App Development Project Report

App Dev Project Report

1. Student Details

Name: Amandeep Singh
Roll Number: 23f2003543
Email: 23f2003543@ds.study.iitm.ac.in
About Me: I am a dual-degree student pursuing **Data Science & Applications** at IIT Madras and **Artificial Intelligence & Machine Learning** at Sir M. Visvesvaraya Institute of Technology. I have a strong interest in artificial intelligence, web development and backend engineering. I enjoy designing full-stack applications, modelling real-world workflows in databases, and building systems that are robust, secure, and easy for end users such as doctors and patients to interact with.

---

2. Project Details

Hospital Management System (HMS)

Problem Statement:

Managing hospital operations manually often results in missed appointments, inconsistent patient records, and inefficient communication between staff, doctors, and patients. There is a need for a unified digital platform that allows users to register, schedule appointments, maintain treatment history, and streamline day-to-day hospital processes efficiently and securely.

Approach:

The application was developed using a modular Flask architecture with SQLAlchemy ORM to manage database operations. A role-based access system was implemented for Admin, Doctor, and Patient, each having distinct functionalities. Appointment booking logic, doctor availability management, and treatment record handling were built through relational models and controller routes. The system follows a clean MVC-like structure, ensuring smooth data flow between models, controllers, and templates while maintaining data consistency and ease of navigation for all user roles.

---

3. AI/LLM Declaration

I used Grok (Grok-4.1) in a limited manner to support the development of specific parts of my project. AI assistance was used to refine certain SQLAlchemy model structures, improve variable naming, and get guidance while implementing the doctor availability–management logic.AI was also used for minor enhancements in inline CSS styling—such as table formatting, button designs, and layout spacing—and for small routing-related suggestions.

The overall usage of AI/LLM was approximately **10–15%**, limited only to guidance and explanation.

All core implementation, end-to-end logic, debugging, and feature integration were done manually.

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework; routing and request handling
SQLite	Lightweight relational database used to store all HMS records
SQLAlchemy ORM	Defines database models and manages object–relational mapping
Flask Session Management	Manual login system with role-based access control (Admin/Doctor/Patient)
Jinja2	Template engine for rendering dynamic HTML pages
HTML5 / CSS3	Frontend structure and styling used across all templates

---

5. Database Schema / ER Diagram

Tables:

1. **Admin** — stores admin user credentials and login details
   (id, email, username, password)

2. **Department** — stores hospital departments and their descriptions
   (id, dep_name, description)

3. **Doctor** — stores doctor profiles and their department mapping
   (id, name, email, username, password, Dep_id, gender, is_blacklisted)

4. **Patient** — stores patient details and login credentials
   (id, name, email, username, password, age, gender, is_blacklisted)

5. **Appointment** — stores appointment bookings between patients and doctors
   (id, p_id, d_id, date, time, status)

6. **Treatment** — stores treatment records linked to specific appointments
   (id, appoint_id, diagnosis, prescription, notes, created_at)

7. **Medicine** — stores medicines prescribed under a specific treatment
   (id, treatment_id, name, dosage, duration)

8. **DoctorAvailability** — stores availability slots for doctors on specific dates
   (id, doctor_id, date, morning_slot, evening_slot)

Relationships:

Relationship Type	From Table	To Table	Description
One-to-Many	Department	Doctor	One department can have many doctors; each doctor belongs to exactly one department (`Doctor.Dep_id → Department.id`).
One-to-Many	Doctor	Appointment	One doctor can have many appointments; each appointment is booked with exactly one doctor (`Appointment.d_id → Doctor.id`).
One-to-Many	Patient	Appointment	One patient can book many appointments; each appointment is associated with one patient (`Appointment.p_id → Patient.id`).
One-to-Many	Appointment	Treatment	One appointment can have multiple treatment records if needed (`Treatment.appoint_id → Appointment.id`).
One-to-Many	Treatment	Medicine	One treatment record can include multiple medicines (`Medicine.treatment_id → Treatment.id`).
One-to-Many	Doctor	DoctorAvailability	One doctor can configure availability for many dates (`DoctorAvailability.doctor_id → Doctor.id`).
One-to-Many (logical)	Patient	Treatment	A patient can have many treatments over time, indirectly via their appointments.
One-to-Many (logical)	Patient	Medicine	Across all treatments, a patient can be prescribed many medicines over multiple visits.

ER Diagram

6. API RESOURCES ENDPOINT

6.1 Public & Authentication Endpoints

Endpoint	Method(s)	Description
/	GET, POST	Renders the home page of the application.
/patient_register	GET, POST	Displays patient registration form and creates a new patient account.
/patient_login	GET, POST	Authenticates a patient using username and password, sets session, redirects to patient dashboard.
/doctor_login	GET, POST	Authenticates a doctor and redirects to doctor dashboard after setting session.
/admin_login	GET, POST	Authenticates an admin and redirects to admin dashboard after setting session.
/logout	GET, POST	Clears the current session and logs out any user (admin/doctor/patient).

6.2 Patient-Side Endpoints

Endpoint	Method(s)	Description
/doctor_dashboard	GET	Displays upcoming booked appointments for the logged-in doctor and a list of all patients they have treated (completed appointments).
/view_patient_history/<int:patient_id>	GET	Allows a doctor to view treatment history of a specific patient, but only for appointments where that doctor was involved and status is Completed.
/complete_appointment/<int:appt_id>	POST	Marks a booked appointment as Completed for the logged-in doctor.
/cancel_appointment/<int:appt_id>	POST	Allows the doctor to cancel one of their appointments.
/update_patient_history/<int:appt_id>	GET, POST	Enables the doctor to add diagnosis, prescription, notes, and prescribed medicines for a given appointment; also marks the appointment as Completed.
/doctor_availability	GET, POST	Lets the logged-in doctor view and update their availability for the next 7 days with morning/evening slot selection.

6.4 Admin-Side Endpoints

Endpoint	Method(s)	Description
/admin_dashboard	GET, POST	Shows an overview of all doctors, patients, and appointments; supports search by patient/doctor name.
/doctor_register	GET, POST	Allows admin to register a new doctor and assign them to a department.

/edit_doctor/<int:doctor_id>	GET, POST	Allows admin to edit doctor details (name, email, username, department, gender).
/edit_patient/<int:patient_id>	GET, POST	Allows admin to edit patient details (name, email, username, age, gender).
/delete_doctor/<int:doctor_id>	GET	Deletes a doctor record from the system.
/delete_patient/<int:patient_id>	GET	Deletes a patient record from the system.
/blacklist_doctor/<int:doctor_id>	GET	Marks a doctor as blacklisted; blacklisted doctors cannot log in.
/unblacklist_doctor/<int:doctor_id>	GET	Removes a doctor from blacklist and restores access.
/blacklist_patient/<int:patient_id>	GET	Marks a patient as blacklisted; blacklisted patients cannot log in.
/unblacklist_patient/<int:patient_id>	GET	Removes a patient from blacklist and restores access.
/view_appointment/<int:appt_id>	GET	Allows admin to view appointment details along with the associated patient, doctor, and completed treatments for that pair.

---

7. Video Presentation

Drive Link:
https://drive.google.com/file/d/1u0zeTgwOMX_n23lUSyl2wESY4bKNwHEp/view?usp=drivesdk